Kevin Küster

Zusammenfassung: Single Inheritance

# Single Inheritance

## OOP auf Maschinenebene

Prinzip zur Wiederverwendung von Code im Rahmen OOP

↳ Kapselung von Code und Daten

⇒ „Globaler Datenraum" wird in Partitionen unterteilt

↳ Struktur der Klasse genügt zur Ableitung von Unterklassen

↳ Code muss nicht kopiert werden

⇒ Unterscheidung von Klassen als statisches Element, Objekte als dynamisches Äquivalent

⇒ Klassen dienen als Partitionierung } Existieren nicht zur Laufzeit ; Struktur

Objekte als Instanziierung zur Laufzeit } Art & Umfang der Daten

Code Methoden ↪ Methodentabelle

↳ Zeiger auf Methodenimplementierung (Code-Speicher [Array])

↳ Jede Methode einer Klasse entspricht einem Eintrag.
enthält Adresse der Implementierung

↳ Aufruf (M.-Name) erfolgt via Indizierung im Array

⇒ Zeiger pro Objekt auf Methodentabelle

## Einfachvererbung

Vererbung erfolgt auf Basis der Partitionierung

↳ Unterliegende Position kann zusätzliche Methoden definieren, erbt alle überliegenden Methoden

⇒ abgeleitete Klasse entspricht unterliegender Partition)

Überschreiben von Methoden: Adressen der Methodentabelle überschreiben

## MI

Code-Hochsprache:

```
class A {                    class B extends A {
  int a1, a2;                  long b1, b2;
  public A() {}                public B() {}
  int m1() {}                  int m1() {}
  boolean m2() {}              long m3() {}
}                            }
```

## Methodentabelle & Feldtabelle

Funktion $T$ : Beschreibt Tabellen $\begin{cases} T_m: \text{Methoden-Tab.} \\ T_f: \text{Feld-Tab.} \end{cases}$

$T_m(A) := m1, m2$ , $T_m(B) := m1, m2, m3$

$T_f(A) := a1, a2$ , $T_f(B) := a1, a2, b1, b2$

## Objektlayout

Objekt wird durch Konstruktoraufruf initialisiert

↳ Objekt wird dynamisch am Heap erzeugt; hp wird um Größe des Objekts plus Maschinenwort für Zeiger auf Methodentabelle erhöht

⇒ Konvention: Erstes Maschinenwort eines Objekts zum Speichern des Methodentabellen-Zeigers (Reserviert)

## Klassen / Methodentabellen

```
A: [
    0: Adresse A::m1
    4: Adresse A::m2
   ]
```

Kodierung nach Ordnung in $T_m(A)$ ; Adressen entsprechen einem Maschinenwort (4 Byte)

```
B: [
    0: Adresse B::m1 ←
    4: Adresse A::m2
    8: Adresse B::m3
   ]
```

„←": Überschreibung von m1 in B

## Objekte

Objekte analog $(T_m, T_f)$:

```
A Objekt: [
    0: Zeiger auf Methodentab. von A
    4: a1 (Integer)
    8: a2 (Integer)
   ]
```

⇒ 12 Byte Speicherplatz

```
B Objekt: [
    0: -"- von B
    4: a1 (Integer)
    8: a2 (Integer)
   12: b1 (long)
   20: b2 (long)
   ]
```

⇒ 28 Byte Speicherplatz

## Methodenaufruf

```
A a = new B();
a.m1();  // B::m1
```

a → B Objekt:

```
[0xCAFE:
   0: Zeiger (Methodentab. B)
   4: a1 (Integer)
   8: a2 (Integer)
  12: b1 (long)
  20: b2 (long)
]
```

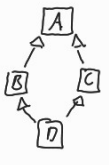
```
[0xDEAD:
  0: Adresse B::m1
  4: Adresse A::m2
  8: Adresse B::m3
]
```

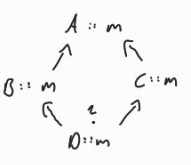
Assembler: i) Dereferenzieren Zeiger in Variable a (0xCAFE)
ii) Auflösen Zeiger an Position 0
iii) Dereferenzieren Zeiger an Position 0 (0xDEAD)
iv) Indizierung Methodentabelle via kodierte Index (hier: 0)

```
⇒ MOVE W a, R0      ← Referenz (a) in R0 speichern
  MOVE W R0, -!SP   ← this Objektzeiger auf Stack legen (erster Parameter)
  MOVE W 0+!R0, R1  ← Methodentabelle in R1
  MOVE W 0+!R1, R2  ← Index in Tab. nach R2
  CALL !R2          ← Adr. dereferenzieren + Aufruf
```

⇒ Methode: Dynamisches Binden

## Problem der Mehrfachvererbung




Keine Konvention, welche überschriebene Methode m D erben soll (B::m oder C::m)

↳ ggf. Mehrfache Methodentabellen

⇒ Hoher Aufwand

**Beispiel** - Konstruktion Methodentabelle (vtable)

```
class Fahrzeug {
    int geschw;
    void starten() { }
    void stoppen() { }
    int getGeschw() { }
}
class Auto extends Fahrzeug {
    int anzTueren;
    void starten() { }
    void hupen() { }
}
```

Fahrzeug:

| Index | M. Name | Adr. |
|---|---|---|
| 0 | starten | → Fahrzeug::starten |
| 4 | stoppen | → Fahrzeug::stoppen |
| 8 | getGeschw | → Fahrzeug::getGeschw |

Auto:

| Index | M. Name | Adr. |
|---|---|---|
| 0 | starten | → Auto::starten |
| 4 | stoppen | → Fahrzeug::stoppen |
| 8 | getGeschw | → Fahrzeug::getGeschw |
| 12 | hupen | → Auto::hupen |

MI:

vtable_Fahrzeug:
```
DD W Fahrzeug_starten
DD W Fahrzeug_stoppen
DD W Fahrzeug_getGeschw
```

MI:

vtable_Auto:
```
DD W Auto_starten
DD W Fahrzeug_stoppen
DD W Fahrzeug_getGeschw
DD W Auto_hupen
```

---

**Beispiel** - Objektlayout

Fahrzeug-Objektlayout:

| Offset | Feld | Typ | Größe |
|---|---|---|---|
| 0 | vtable-Zeiger | Pointer | 4 Bytes |
| 4 | geschwindigkeit | Integer | 4 Bytes |
| Gesamt | | | 8 Bytes |

Auto-Objektlayout:

| Offset | Feld | Typ | Größe |
|---|---|---|---|
| 0 | vtable-Zeiger | Pointer | 4 Bytes |
| 4 | geschwindigkeit | Integer | 4 Bytes |
| 8 | anzahlTueren | Integer | 4 Bytes |
| Gesamt | | | 12 Bytes |

MI: Auto_new:
```
PUSHR
MOVE W SP, R13
;ALLOZIERE 12 Bytes auf Heap
MOVE W hp, R0
ADD W hp, I 12, hp
;vtable-Zeiger setzen
MOVE vtable_Auto, 0+!R0
;init Geschw. = 0
MOVE W I 0, 4+!R0
;init anzTueren = 4
MOVE W I 4, 8+!R0
;Rückgabe
MOVE W R0, 64+!R13
MOVE W R13, SP
POPR
RET
```

---

**Beispiel** - Methodenaufruf

```
Fahrzeug f = new Auto();
f.starten();
```

MI:
```
; Objekterzeugung
CLEAR W -!SP ; Platz Rückgabewert
CALL Auto_new ; Auto-Obj erzeugen
MOVE W !SP+, f ; Obj.-Adr. in f speichern
;f.starten
MOVE W f, R0
MOVE W R0, -!SP
MOVE W 0+!R0, R1 ; vtable-Zeiger
MOVE W 0+!R1, R2 ; R2 := Auto::starten (Adr.)
CALL !R2
ADD W SP, I 4, SP ; this-Zeiger entfernen
```